

HTML & CSS LAB (BCS-155)

B.Tech CSE 1st YEAR

SUBMITTED TO:

AYUSH KUMAR YADAV

SUBMITTED BY:

AMAN PAL
ROLL NO: BCS2026458

INDEX

SN.	PROGRAM NAME	PAGE	DATE	SIGNATURE
1	Basic Structure of an HTML Document and Text Formatting	1		
2	Ordered, Unordered, Nested and Description Lists	3		
3	Hyperlinks, Images and Page Navigation	5		
4	HTML Tables with Rowspan and Colspan	7		
5	Registration Form using HTML Form Elements	9		
6	Semantic HTML5 Page Layout	11		
7	Inline, Internal and External CSS	13		
8	CSS Selectors (Element, Class, ID, Group and Descendant)	15		
9	CSS Box Model (Margin, Border and Padding)	17		
10	Text, Font, Color and Background Styling	19		
11	Horizontal Navigation Menu with Hover Effect	21		
12	Page Layout using CSS Flexbox	23		
13	Page Layout using CSS Grid	25		
14	Responsive Web Page using Media Queries	27		
15	Mini Project: Personal Portfolio Web Page	29		

EXPERIMENT NO. 1

1. Experiment Title:
Basic Structure of an HTML Document and Text Formatting

2. Aim:
To create a simple web page using the basic HTML tags: headings, paragraphs, text formatting, line break and horizontal rule.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

4. Theory:
HTML (HyperText Markup Language) is the standard language for creating web pages. Every document starts with the DOCTYPE declaration and has the <html>, <head> and <body> elements. Headings (h1-h6) define titles, <p> defines a paragraph, and tags such as <b>, <i>, <u>, <sub> and <sup> format text.

5. Algorithm / Procedure:
1. Open a text editor and create a file named index.html.
2. Write the DOCTYPE, html, head (with meta charset and title) and body tags.
3. Add headings, paragraphs and formatting tags inside the body.
4. Save the file and open it in the web browser.
5. Observe the output.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Welcome to HTML Lab</h1>
  <h2>Invertis University, Bareilly</h2>
  <p>This is a <b>bold</b>, <i>italic</i> and <u>underlined</u> text.</p>
  <p>H<sub>2</sub>O and x<sup>2</sup> are subscript and superscript.</p>
  <hr>
  <p>Line one<br>Line two after a line break.</p>
  <pre>Preformatted text
    keeps   its    spaces.</pre>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "Basic Structure of an HTML Document and Text Formatting" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 2

1. Experiment Title:
Ordered, Unordered, Nested and Description Lists

2. Aim:
To create different types of lists in HTML.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

4. Theory:
HTML supports ordered lists (<ol>), unordered lists (<ul>) and description lists (<dl>). Each item of <ol> and <ul> is written with <li>. Lists can be nested by placing a list inside an <li>. The type attribute changes the numbering style.

5. Algorithm / Procedure:
1. Create a new HTML file.
2. Create an ordered list of subjects using <ol> and <li>.
3. Create an unordered list with a nested list inside it.
4. Create a description list using <dl>, <dt> and <dd>.
5. Save and open the file in the browser.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head><title>Lists</title></head>
<body>
  <h2>Ordered List</h2>
  <ol type="1">
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
  </ol>

  <h2>Unordered List with Nested List</h2>
  <ul>
    <li>Fruits
      <ul>
        <li>Apple</li>
        <li>Mango</li>
      </ul>
    </li>
    <li>Vegetables</li>
  </ul>

  <h2>Description List</h2>
  <dl>
    <dt>HTML</dt>
    <dd>Language used to create the structure of web pages.</dd>
    <dt>CSS</dt>
    <dd>Language used to style web pages.</dd>
  </dl>
</body>
</html>
```

7. Input:
No input required.

8. Output:

Ordered List

1. HTML
2. CSS
3. JavaScript

Unordered List with Nested List

- Fruits
  - Apple
  - Mango
- Vegetables

Description List

HTML
: Language used to create the structure of web pages.

CSS
: Language used to style web pages.

9. Result:

The experiment "Ordered, Unordered, Nested and Description Lists" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 3

1. Experiment Title:
Hyperlinks, Images and Page Navigation

2. Aim:
To create hyperlinks (internal, external and email) and insert an image in a web page.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad, an image file (photo.svg).

4. Theory:
The anchor tag <a> with the href attribute creates a hyperlink. A link can point to another website, to a section of the same page (using id and #) or to an email address (mailto:). The <img> tag inserts an image using the src attribute and shows the alt text if the image cannot be loaded.

5. Algorithm / Procedure:
1. Create an HTML file and keep the image photo.svg in the same folder.
2. Add an external link using <a href="https://...">.
3. Add an internal link to a section using an id.
4. Add a mailto link.
5. Insert the image using <img> with width, height and alt attributes.

6. Program / Queries / Commands:
File: links.html

```
<!DOCTYPE html>
<html>
<head><title>Links and Images</title></head>
<body>
  <h1 id="top">Links and Images</h1>
  <p><a href="https://www.invertis.org" target="_blank">Visit Invertis University</a></p>
  <p><a href="#bottom">Go to bottom of the page</a></p>
  <p><a href="mailto:student@example.com">Send an email</a></p>

  <img src="photo.svg" alt="Sample picture" width="300" height="180">

  <p id="bottom"><a href="#top">Back to top</a></p>
</body>
</html>
```

7. Input:
No input required.

8. Output:

Links and Images

Visit Invertis University

Go to bottom of the page

Send an email



Back to top

9. Result:

The experiment "Hyperlinks, Images and Page Navigation" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 4

1. Experiment Title:
HTML Tables with Rowspan and Colspan

2. Aim:
To create a table with a caption, header cells, and merged rows and columns.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

4. Theory:
A table is created with <table>, rows with <tr>, header cells with <th> and data cells with <td>. The colspan attribute merges cells across columns and rowspan merges cells across rows. <caption> gives a title to the table and the border attribute shows the cell borders.

5. Algorithm / Procedure:
1. Create an HTML file and add a <table> with border="1".
2. Add a <caption> and a header row using <th>.
3. Add data rows using <tr> and <td>.
4. Use colspan and rowspan to merge cells.
5. Save and open the file in the browser.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head><title>Table</title></head>
<body>
  <table border="1" cellpadding="8" cellspacing="0">
    <caption><b>Class Time Table</b></caption>
    <tr>
      <th>Day</th><th>9:00 - 10:00</th><th>10:00 - 11:00</th><th>11:00 - 12:00</th>
    </tr>
    <tr>
      <td>Monday</td><td>Maths</td><td>Physics</td><td rowspan="2">Lab</td>
    </tr>
    <tr>
      <td>Tuesday</td><td>English</td><td>Programming</td>
    </tr>
    <tr>
      <td>Wednesday</td><td colspan="3" align="center">Seminar</td>
    </tr>
  </table>
</body>
</html>
```

7. Input:
No input required.

8. Output:

Class Time Table

Day	9:00 - 10:00	10:00 - 11:00	11:00 - 12:00
Monday	Maths	Physics	Lab
Tuesday	English	Programming	
Wednesday	Seminar		

9. Result:

The experiment "HTML Tables with Rowspan and Colspan" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 5

1. Experiment Title:
Registration Form using HTML Form Elements

2. Aim:
To design a student registration form using different form controls.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

4. Theory:
The <form> element collects user input. The <input> tag has many types such as text, email, password, radio, checkbox, date and submit. <select> creates a drop-down list, <textarea> a multi-line box, and <label> describes a control. The attributes required and placeholder help in basic validation and guidance.

5. Algorithm / Procedure:
1. Create an HTML file and add a <form> tag.
2. Add text, email and password fields with labels.
3. Add radio buttons for gender and checkboxes for hobbies.
4. Add a drop-down list for the course and a textarea for the address.
5. Add Submit and Reset buttons and test the form.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head><title>Registration Form</title></head>
<body>
  <h2>Student Registration</h2>
  <form action="#" method="post">
    <p>Name: <input type="text" name="name" placeholder="Full name" required></p>
    <p>Email: <input type="email" name="email" required></p>
    <p>Password: <input type="password" name="pwd" required></p>
    <p>Date of Birth: <input type="date" name="dob"></p>
    <p>Gender:
      <input type="radio" name="g" value="M"> Male
      <input type="radio" name="g" value="F"> Female
    </p>
    <p>Hobbies:
      <input type="checkbox" checked> Reading
      <input type="checkbox"> Music
      <input type="checkbox"> Sports
    </p>
    <p>Course:
      <select name="course">
        <option>B.Tech CSE</option>
        <option>B.Tech ECE</option>
        <option>BCA</option>
      </select>
    </p>
    <p>Address:<br><textarea rows="3" cols="35"></textarea></p>
    <input type="submit" value="Register">
    <input type="reset">
  </form>
</body>
</html>
```

7. Input:
Name, email, password, date of birth, gender, hobbies, course and address entered by the user.

8. Output:



9. Result:

The experiment "Registration Form using HTML Form Elements" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 6

1. Experiment Title:
Semantic HTML5 Page Layout

2. Aim:
To create a web page layout using semantic HTML5 elements.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

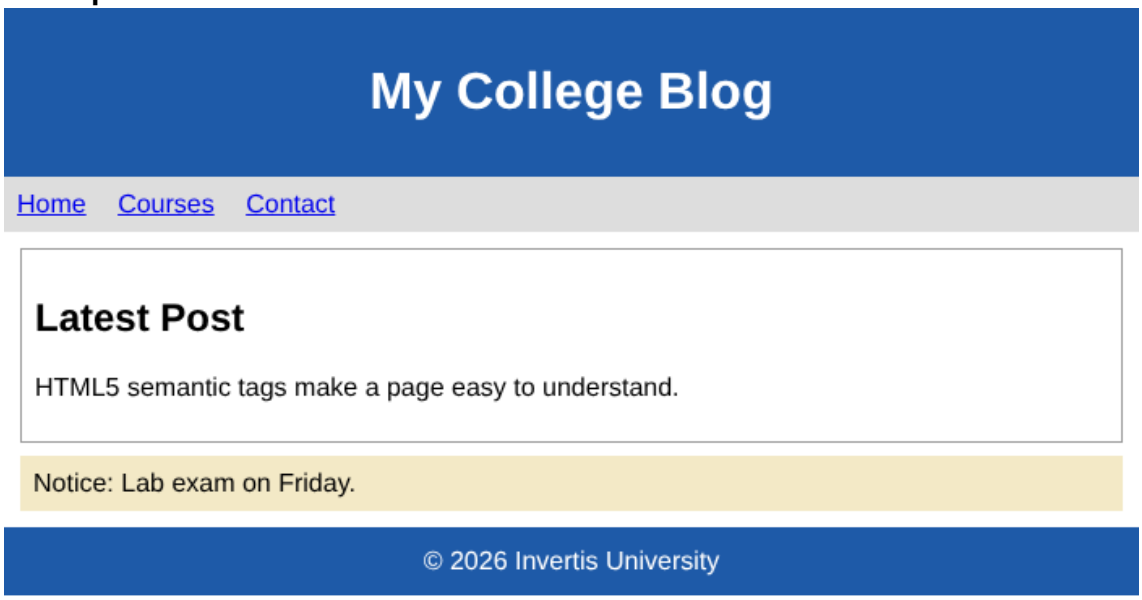
4. Theory:
Semantic elements clearly describe their meaning to the browser and the developer. <header> holds the heading, <nav> the navigation links, <main> the main content, <section> and <article> group related content, <aside> holds side content and <footer> the footer information. They improve readability, accessibility and SEO.

5. Algorithm / Procedure:
1. Create an HTML file.
2. Add <header> and <nav> with links.
3. Add <main> containing <section> and <article>.
4. Add <aside> and <footer>.
5. Apply a light style so that the structure is visible and check the output.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <title>Semantic Layout</title>
  <style>
    body { font-family: Arial; margin: 0; }
    header, footer { background: #1e5aa8; color: white; padding: 12px; text-align: center; }
    nav { background: #ddd; padding: 8px; }
    nav a { margin-right: 15px; }
    main { padding: 10px; }
    section { border: 1px solid #999; padding: 8px; margin-bottom: 8px; }
    aside { background: #f3e9c6; padding: 8px; }
  </style>
</head>
<body>
  <header><h1>My College Blog</h1></header>
  <nav><a href="#">Home</a> <a href="#">Courses</a> <a href="#">Contact</a></nav>
  <main>
    <section>
      <h2>Latest Post</h2>
      <article><p>HTML5 semantic tags make a page easy to understand.</p></article>
    </section>
    <aside>Notice: Lab exam on Friday.</aside>
  </main>
  <footer>&copy; 2026 Invertis University</footer>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "Semantic HTML5 Page Layout" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 7

1. Experiment Title:
Inline, Internal and External CSS

2. Aim:
To apply CSS to a web page using the inline, internal and external methods.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

4. Theory:
CSS (Cascading Style Sheets) controls the presentation of HTML. Inline CSS uses the style attribute of an element, internal CSS is written inside the <style> tag in the head, and external CSS is stored in a separate .css file linked using <link rel="stylesheet">. When styles conflict, the inline style has the highest priority, followed by internal/external styles in the order they are written.

5. Algorithm / Procedure:
1. Create css.html and style.css in the same folder.
2. Write the CSS for h1 in the external file style.css.
3. Link the file using the <link> tag.
4. Add an internal <style> block for the paragraph.
5. Add an inline style to one heading and compare the priorities.

6. Program / Queries / Commands:
File: css.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Types of CSS</title>
  <link rel="stylesheet" href="style.css">
  <style>
    p { color: darkgreen; font-size: 18px; }
  </style>
</head>
<body>
  <h1>External CSS Heading</h1>
  <p>This paragraph is styled by internal CSS.</p>
  <h2 style="color: crimson; text-decoration: underline;">Inline CSS Heading</h2>
</body>
</html>
```

File: style.css

```
h1 {
  color: white;
  background-color: #1e5aa8;
  padding: 10px;
  text-align: center;
}
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "Inline, Internal and External CSS" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 8

1. Experiment Title:
CSS Selectors (Element, Class, ID, Group and Descendant)

2. Aim:
To understand and use different CSS selectors.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

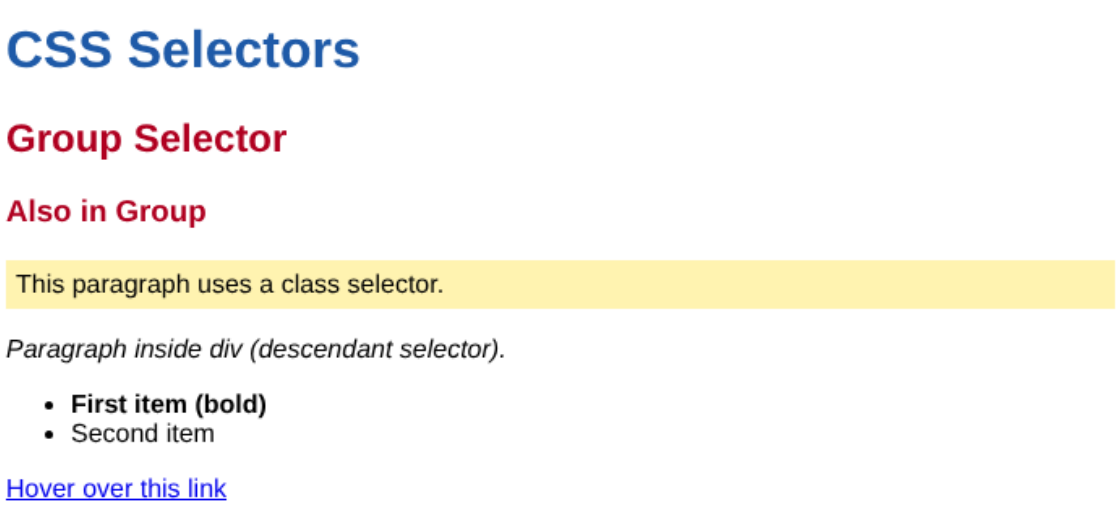
4. Theory:
A selector chooses the HTML elements to style. An element selector uses the tag name, a class selector uses .name, an ID selector uses #name (unique in a page), a grouping selector separates selectors with commas, a descendant selector uses a space (div p), and a pseudo-class such as :hover or :first-child selects a special state or position.

5. Algorithm / Procedure:
1. Create an HTML file with a heading, paragraphs, a div and a list.
2. Write an element selector for the body and headings.
3. Write class (.note) and ID (#main-title) selectors.
4. Write grouping, descendant and :first-child selectors.
5. Open the page and verify that each rule is applied.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <title>Selectors</title>
  <style>
    body { font-family: Arial; }
    #main-title { color: #1e5aa8; }
    .note { background: #fff3b0; padding: 6px; }
    h2, h3 { color: #b00020; }
    div p { font-style: italic; }
    li:first-child { font-weight: bold; }
    a:hover { color: orange; }
  </style>
</head>
<body>
  <h1 id="main-title">CSS Selectors</h1>
  <h2>Group Selector</h2>
  <h3>Also in Group</h3>
  <p class="note">This paragraph uses a class selector.</p>
  <div><p>Paragraph inside div (descendant selector).</p></div>
  <ul><li>First item (bold)</li><li>Second item</li></ul>
  <a href="#">Hover over this link</a>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "CSS Selectors (Element, Class, ID, Group and Descendant)" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 9

1. Experiment Title:
CSS Box Model (Margin, Border and Padding)

2. Aim:
To understand the CSS box model by applying margin, border and padding to elements.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

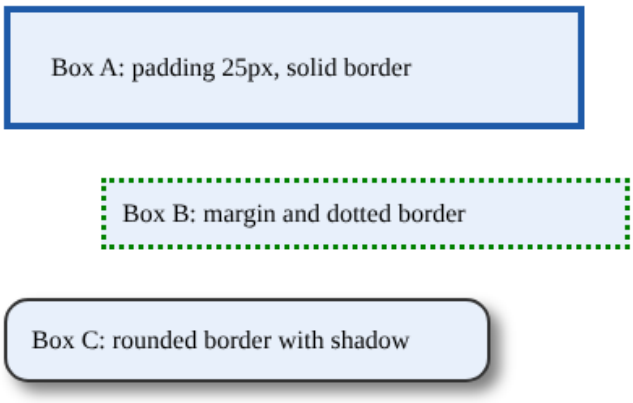
4. Theory:
Every HTML element is a rectangular box made of content, padding, border and margin. Padding is the space between content and border, border surrounds the padding, and margin is the space outside the border. The total width of a box is width + padding + border + margin unless box-sizing: border-box is used.

5. Algorithm / Procedure:
1. Create an HTML file with three div elements.
2. Give the first div a fixed width, padding and border.
3. Give the second div a margin and a dotted border.
4. Give the third div border-radius and box-shadow.
5. Observe the spacing in the browser.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <title>Box Model</title>
  <style>
    div { width: 300px; background: #e8f0fb; }
    .a { padding: 25px; border: 4px solid #1e5aa8; }
    .b { margin: 30px 0 30px 60px; padding: 10px; border: 3px dotted green; }
    .c { padding: 15px; border: 2px solid #333; border-radius: 15px;
         box-shadow: 6px 6px 8px gray; box-sizing: border-box; }
  </style>
</head>
<body>
  <div class="a">Box A: padding 25px, solid border</div>
  <div class="b">Box B: margin and dotted border</div>
  <div class="c">Box C: rounded border with shadow</div>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "CSS Box Model (Margin, Border and Padding)" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 10

1. Experiment Title:
Text, Font, Color and Background Styling

2. Aim:
To style text and backgrounds using CSS font, text and background properties.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

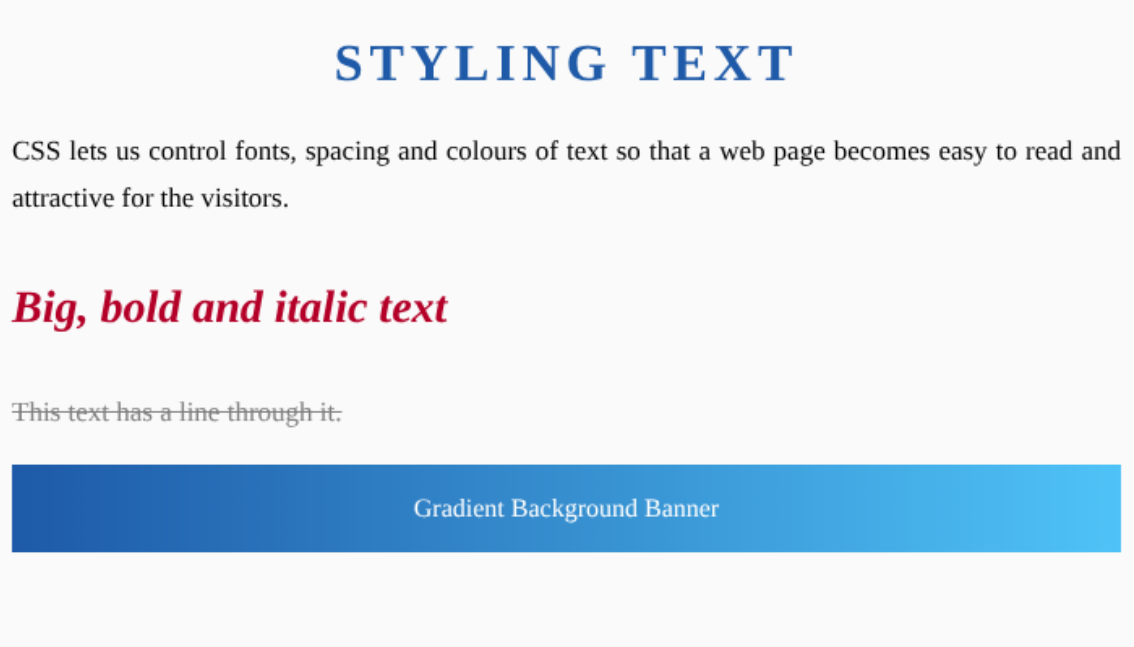
4. Theory:
CSS font properties include font-family, font-size, font-weight and font-style. Text properties include color, text-align, text-decoration, text-transform, letter-spacing and line-height. Colors can be given as names, hex codes, rgb() or hsl(). The background-color and linear-gradient() set the background of an element.

5. Algorithm / Procedure:
1. Create an HTML file with headings and paragraphs.
2. Set font family, size and weight for the body and headings.
3. Apply text-align, text-transform and letter-spacing.
4. Set a background colour and a gradient for a section.
5. Open the file and check the result.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <title>Text Styling</title>
  <style>
    body { font-family: Georgia, serif; background-color: #fafafa; }
    h1 { text-align: center; text-transform: uppercase;
         letter-spacing: 4px; color: #1e5aa8; }
    p { font-size: 17px; line-height: 1.7; text-align: justify; }
    .big { font-size: 28px; font-weight: bold; font-style: italic; color: rgb(180,0,40); }
    .banner { background: linear-gradient(to right, #1e5aa8, #4fc3f7);
              color: white; padding: 18px; text-align: center; }
    .strike { text-decoration: line-through; color: gray; }
  </style>
</head>
<body>
  <h1>Styling Text</h1>
  <p>CSS lets us control fonts, spacing and colours of text so that a web page
     becomes easy to read and attractive for the visitors.</p>
  <p class="big">Big, bold and italic text</p>
  <p class="strike">This text has a line through it.</p>
  <div class="banner">Gradient Background Banner</div>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "Text, Font, Color and Background Styling" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 11

1. Experiment Title:
Horizontal Navigation Menu with Hover Effect

2. Aim:
To create a horizontal navigation bar using an unordered list and CSS.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

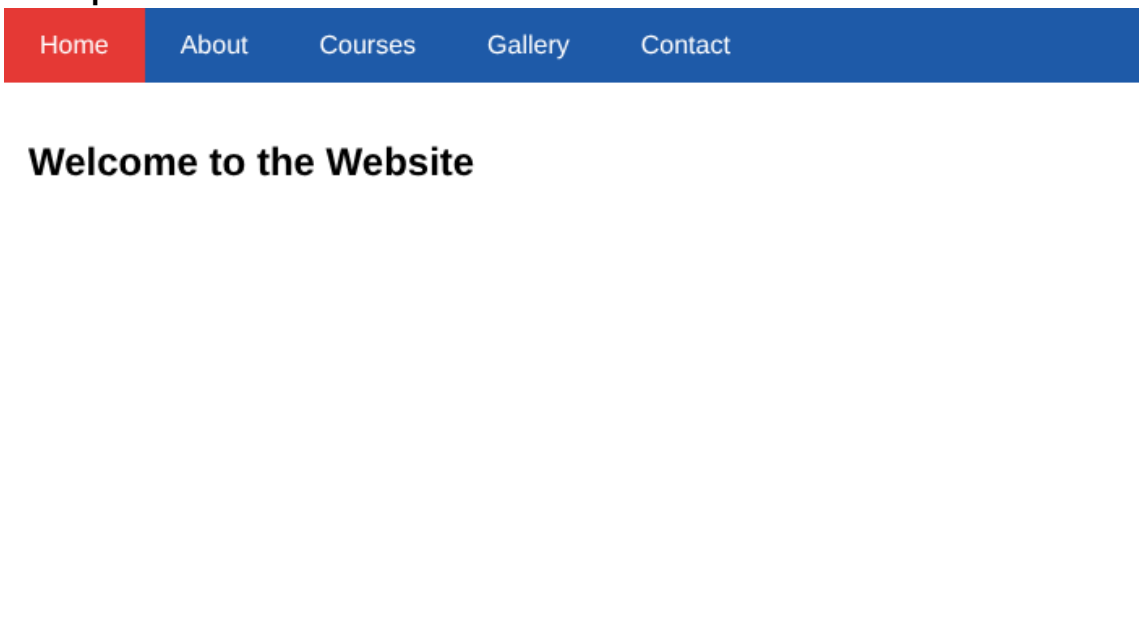
4. Theory:
A navigation menu is usually built from an unordered list of links. CSS removes the bullets (list-style: none), makes the items sit side by side (display: inline-block or float) and styles the links with padding and colours. The :hover pseudo-class changes the style when the mouse is over a link.

5. Algorithm / Procedure:
1. Create a <nav> containing a <ul> with five <li> links.
2. Remove the list bullets and margins.
3. Display the items in one line and style the links.
4. Add a hover colour and highlight the active link.
5. Test the menu in the browser.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <title>Navigation Menu</title>
  <style>
    body { margin: 0; font-family: Arial; }
    ul { list-style: none; margin: 0; padding: 0; background: #1e5aa8; overflow: hidden; }
    li { float: left; }
    li a { display: block; color: white; padding: 14px 22px; text-decoration: none; }
    li a:hover { background: #0f3d7a; }
    li a.active { background: #e53935; }
  </style>
</head>
<body>
  <ul>
    <li><a class="active" href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Courses</a></li>
    <li><a href="#">Gallery</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
  <h2 style="padding:15px">Welcome to the Website</h2>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "Horizontal Navigation Menu with Hover Effect" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 12

1. Experiment Title:
Page Layout using CSS Flexbox

2. Aim:
To create a flexible layout and card row using CSS Flexbox.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

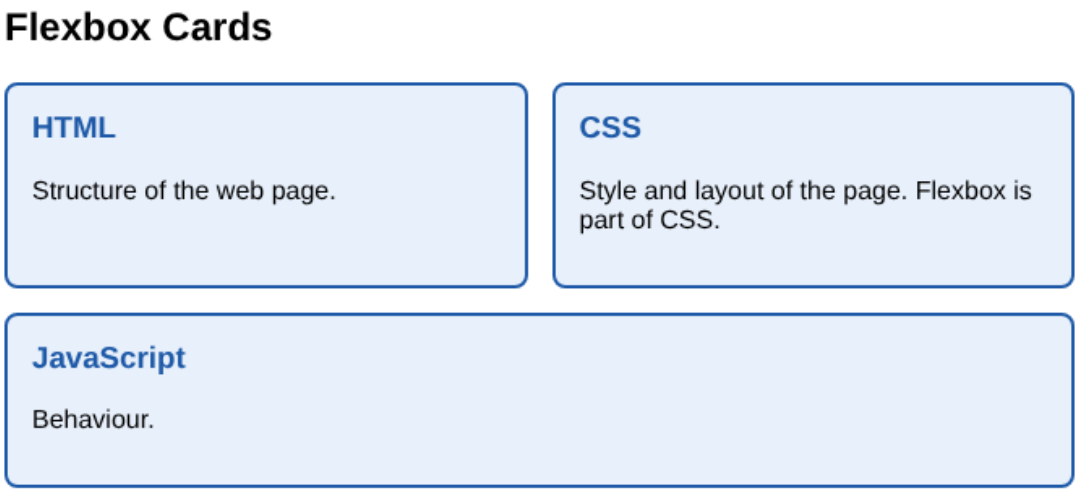
4. Theory:
Flexbox is a one-dimensional layout model. Setting display: flex on a container makes its children flex items. The properties flex-direction, justify-content, align-items, flex-wrap and gap control the direction, alignment and spacing of the items, and flex: 1 lets an item grow to share the free space.

5. Algorithm / Procedure:
1. Create a container div with three card divs inside.
2. Apply display: flex and gap to the container.
3. Use justify-content and align-items to align the cards.
4. Use flex: 1 so that the cards share the width equally.
5. Add flex-wrap and verify the output.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <title>Flexbox</title>
  <style>
    body { font-family: Arial; margin: 20px; }
    .container { display: flex; justify-content: space-between;
                 align-items: stretch; gap: 15px; flex-wrap: wrap; }
    .card { flex: 1; min-width: 180px; background: #e8f0fb;
            border: 2px solid #1e5aa8; border-radius: 8px; padding: 15px; }
    .card h3 { margin-top: 0; color: #1e5aa8; }
  </style>
</head>
<body>
  <h2>Flexbox Cards</h2>
  <div class="container">
    <div class="card"><h3>HTML</h3><p>Structure of the web page.</p></div>
    <div class="card"><h3>CSS</h3><p>Style and layout of the page. Flexbox is part of CSS.</p></div>
    <div class="card"><h3>JavaScript</h3><p>Behaviour.</p></div>
  </div>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "Page Layout using CSS Flexbox" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 13

1. Experiment Title:
Page Layout using CSS Grid

2. Aim:
To design a web page layout using CSS Grid.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

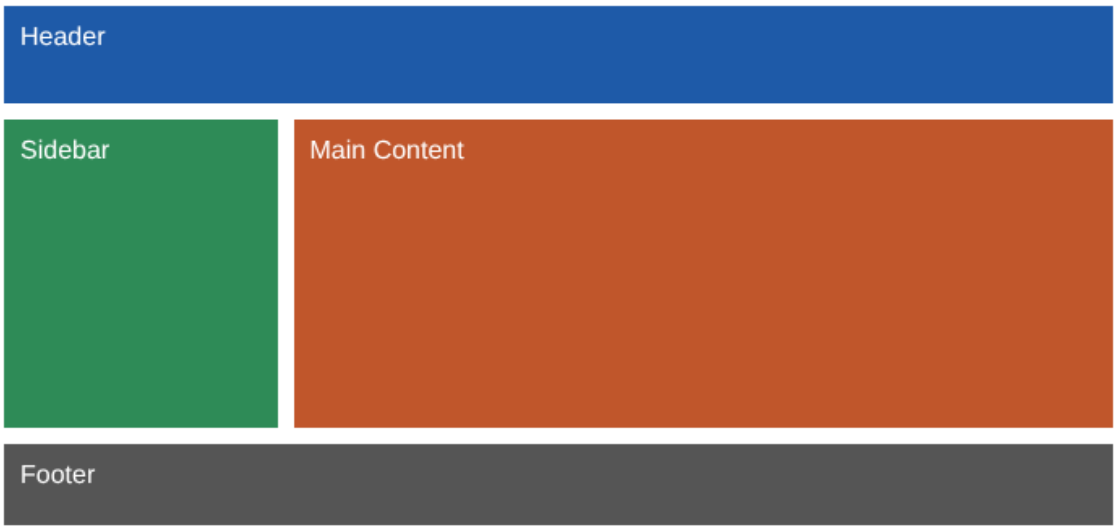
4. Theory:
CSS Grid is a two-dimensional layout system. A container with display: grid defines columns and rows using grid-template-columns and grid-template-rows. The fr unit divides the free space, gap sets spacing, and grid-template-areas with grid-area names the regions of the layout (header, sidebar, content, footer).

5. Algorithm / Procedure:
1. Create a container with header, sidebar, content and footer divs.
2. Apply display: grid with grid-template-columns and rows.
3. Name the areas using grid-template-areas.
4. Assign each div to its area using grid-area.
5. Open the page and check the layout.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Grid</title>
  <style>
    .layout { display: grid; gap: 10px; height: 320px;
      grid-template-columns: 1fr 3fr;
      grid-template-rows: 60px 1fr 50px;
      grid-template-areas: "head head"
                           "side main"
                           "foot foot"; }
    .layout div { padding: 10px; color: white; font-family: Arial; }
    .head { grid-area: head; background: #1e5aa8; }
    .side { grid-area: side; background: #2e8b57; }
    .main { grid-area: main; background: #c0562b; }
    .foot { grid-area: foot; background: #555; }
  </style>
</head>
<body>
  <div class="layout">
    <div class="head">Header</div>
    <div class="side">Sidebar</div>
    <div class="main">Main Content</div>
    <div class="foot">Footer</div>
  </div>
</body>
</html>
```

7. Input:
No input required.

8. Output:



9. Result:

The experiment "Page Layout using CSS Grid" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 14

1. Experiment Title:
Responsive Web Page using Media Queries

2. Aim:
To create a responsive web page that adjusts its layout to the screen size.

3. Software / Tools Required:
Web browser (Chrome) with Developer Tools, VS Code / Notepad.

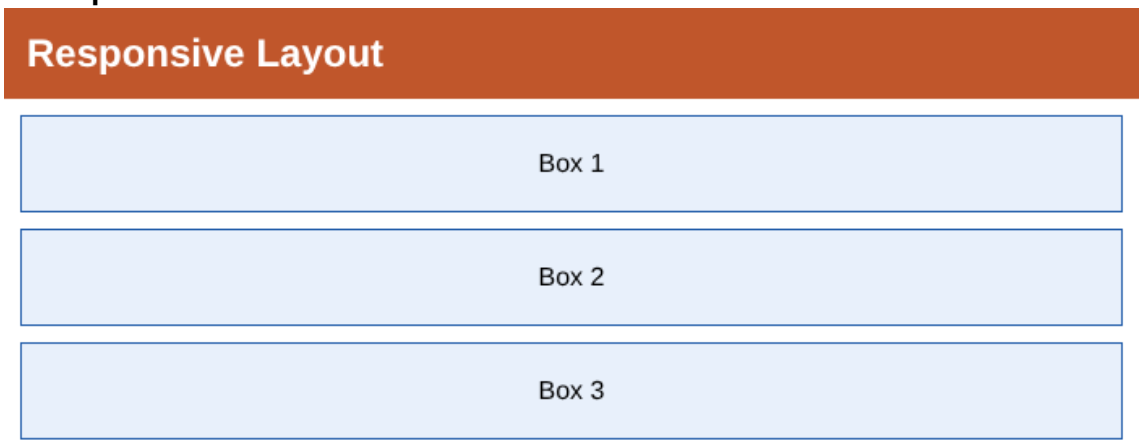
4. Theory:
A responsive page adapts to desktops, tablets and mobile phones. The viewport meta tag sets the page width to the device width. Media queries (@media) apply CSS only when a condition, such as max-width, is true. Relative units (%, rem, fr) and flexible layouts help the content resize smoothly.

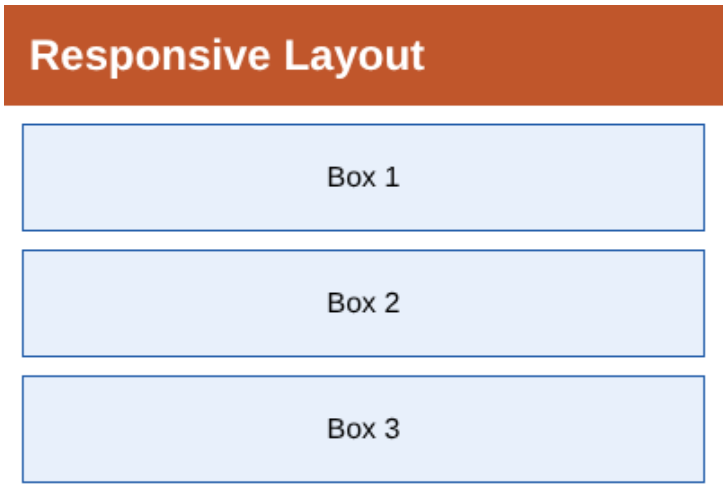
5. Algorithm / Procedure:
1. Add the viewport meta tag in the head.
2. Create a three-column layout using CSS Grid.
3. Write a media query for screens up to 700px wide that shows one column.
4. Open the page in the browser and resize the window.
5. Observe the change from three columns to one column.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Responsive Page</title>
  <style>
    body { font-family: Arial; margin: 0; }
    h2 { background: #1e5aa8; color: white; margin: 0; padding: 14px; }
    .row { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; padding: 10px; }
    .box { background: #e8f0fb; border: 1px solid #1e5aa8; padding: 20px; text-align: center; }
    @media (max-width: 700px) {
      .row { grid-template-columns: 1fr; }
      h2 { background: #c0562b; }
    }
  </style>
</head>
<body>
  <h2>Responsive Layout</h2>
  <div class="row">
    <div class="box">Box 1</div>
    <div class="box">Box 2</div>
    <div class="box">Box 3</div>
  </div>
</body>
</html>
```

7. Input:
Browser window width: desktop (700px) and mobile (400px).

8. Output:





9. Result:

The experiment "Responsive Web Page using Media Queries" was performed successfully. The web page was created, opened in the browser and the output was verified.

EXPERIMENT NO. 15

1. Experiment Title:
Mini Project: Personal Portfolio Web Page

2. Aim:
To design a complete personal portfolio page using HTML and CSS concepts learned in the lab.

3. Software / Tools Required:
Web browser (Chrome), VS Code / Notepad.

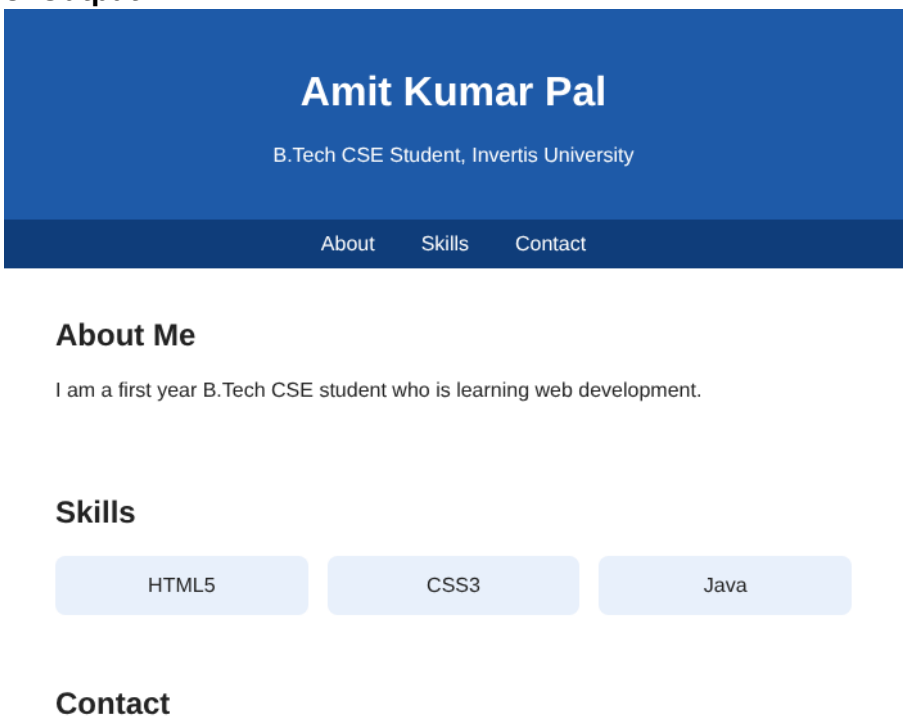
4. Theory:
A complete web page combines semantic HTML (header, nav, section, footer), CSS selectors, the box model, Flexbox/Grid layout, a navigation bar, a form and hover effects. Planning the layout first and keeping the CSS organised makes the page easy to maintain.

5. Algorithm / Procedure:
1. Plan the sections: header, about, skills, contact and footer.
2. Write the semantic HTML structure.
3. Style the header, navigation bar and sections using CSS.
4. Use Flexbox for the skill cards and a form for the contact section.
5. Test the page in the browser and fix any layout problems.

6. Program / Queries / Commands:
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Portfolio - Amit Kumar Pal</title>
  <style>
    * { box-sizing: border-box; }
    body { margin: 0; font-family: Arial, sans-serif; color: #222; }
    header { background: #1e5aa8; color: white; text-align: center; padding: 25px; }
    nav { background: #0f3d7a; text-align: center; }
    nav a { color: white; display: inline-block; padding: 10px 18px; text-decoration: none; }
    nav a:hover { background: #e53935; }
    section { padding: 18px 40px; }
    .skills { display: flex; gap: 15px; }
    .skill { flex: 1; text-align: center; padding: 14px; background: #e8f0fb;
             border-radius: 8px; transition: transform .3s; }
    .skill:hover { transform: translateY(-6px); }
    input, textarea { width: 100%; padding: 8px; margin-bottom: 8px; }
    button { background: #1e5aa8; color: white; border: 0; padding: 9px 22px; }
    footer { background: #333; color: white; text-align: center; padding: 10px; }
  </style>
</head>
<body>
  <header><h1>Amit Kumar Pal</h1><p>B.Tech CSE Student, Invertis University</p></header>
  <nav><a href="#about">About</a><a href="#skills">Skills</a><a href="#contact">Contact</a></nav>
  <section id="about"><h2>About Me</h2>
    <p>I am a first year B.Tech CSE student who is learning web development.</p></section>
  <section id="skills"><h2>Skills</h2>
    <div class="skills">
      <div class="skill">HTML5</div><div class="skill">CSS3</div><div class="skill">Java</div>
    </div></section>
  <section id="contact"><h2>Contact</h2>
    <form><input type="text" placeholder="Your name">
      <textarea rows="2" placeholder="Message"></textarea>
      <button type="button">Send</button></form></section>
  <footer>&copy; 2026 Amit Kumar Pal</footer>
</body>
</html>
```

7. Input:
Name and message typed in the contact form (optional).

8. Output:



9. Result:
The experiment "Mini Project: Personal Portfolio Web Page" was performed successfully. The web page was created, opened in the browser and the output was verified.